

Tema 1. Introducción

Tema 1.1 CC de la computación. Máquinas
secuenciales. Autómatas.



INDICE

- 1. Historia de la CC de la Computación.**
- 2. Conceptos básicos. Funciones algorítmicas.**
- 3. Alfabetos, palabras y lenguajes.**



Historia de la computación a través de sus protagonistas

Temas de trabajo. Bloque 1

1. Alan Turing. Vida y relación con la computación.
2. Stephen Kleene. Vida y obra.
3. Von Neumann. Vida y obra.
4. Noam Chomsky. Vida y obra.
5. Grace Murray Hopper. Vida y obra.
6. Ada Byron. Vida y relación con la programación,
7. Alfred Aho. A la caza del dragón.
8. Kernigan y Ritchie. Vida y relación con la programación.

Temas de trabajo. Bloque 2

1. Hedy Lamarr. Vida y contribuciones a las TIC
2. Evelyn Berezin. Vida, procesadores y vuelos.
3. Frances E. Allen . Vida y obra
4. Anita Borg. Vida y obra.
5. Top Secret Rosies (6) . La construcción de ENIAC.
6. Lynn Conway. Vida y microchips.
7. Jude Milhon. Vida y ciberpunk.
8. Ángela Ruíz Robles. La docencia, tecnología y ergonomía.

Temas de trabajo. Bloque 3

- Personaje a elegir por cada grupo.
- Distinto a los de los otros bloques.
- Se valorará: la originalidad y la "complementariedad" con los de su grupo.

Fecha de entrega: 30/09

Datos imprescindibles:

1. Fecha y lugar de nacimiento y muerte
2. Aportación principal (pueden ser varias)
3. Relación de sus aportaciones con "el presente" (o el futuro previsible)
4. Referencias de donde se ha extraído la información y donde se puede acudir para saber más.

Entregables y principios de diseño:

1. Un informe escrito y un vídeo con una presentación.
2. En la presentación "deben aparecer" todos los miembros del equipo.
3. Su duración es de un máximo de 2 minutos por personaje.



1. Conceptos Básicos

1. Teoría de conjuntos
2. Funciones algorítmicas



0. Preliminares

- Modelo de Demostraciones
 - Todos los teoremas que aparezcan, deben ser demostrados formalmente.
 - A veces, se obvia esta demostración, pero **existe**.
- A veces, solo demostramos que “existe” una solución, pero no sabemos cual.
 - El horizonte nunca se alcanza, pero existe.
- Piezas importantes de una teoría:
 - Axioma.
 - Definición.
 - Teorema, proposición, lema, corolario.



0. Preliminares

- Clasificación de demostraciones:
 - Demostraciones **directas**
 - Sólo utilizan definiciones y teoremas anteriores
 - Demostraciones **constructivas**
 - Construyen herramientas propias para llegar a una conclusión.
 - Demostraciones por **reducción al absurdo**
 - Muestran una contradicción en la negación de lo que se quiere demostrar
 - Demostraciones por **inducción**
 - Sirven para probar propiedades para un número infinito de casos
 - Si se cumple caso inicial, y se cumple caso $n \rightarrow$ Se debe cumplir caso $n+1$
 - Se concluye que se cumple para todo n



1. Teoría de Conjuntos

- Definición 1.1:
 - Un conjunto es una colección de objetos, llamados **elementos**
- Notación:
 - $A = \{a, b, c\}$
 - $W = \{x : x \text{ es un entero positivo}\}$
 - $\{\} = \varphi$ (Conjunto vacío)
- Se define el conjunto de los números naturales como $\mathbb{N}$ al conjunto:
 - $\mathbb{N} = \{0, 1, 2, 3, 4, \dots\}$ (Construcción de los naturales de Von Newman)
 - y
 - $\mathbb{N}^+ = \{1, 2, 3, 4, 5, \dots\}$



1. Teoría de Conjuntos

- Operaciones entre conjuntos:

Sean X, A, B conjuntos y x, a, b elementos

$x \in X: \Leftrightarrow x$ es un elemento de X

$x \notin X: \Leftrightarrow x$ no es un elemento de X

$A \cup B$ (A unión B) = $\{x: x \in A \vee x \in B\}$

$A \cap B$ (A intersección B) = $\{x: x \in A \wedge x \in B\}$

$A \setminus B$ (A menos B) = $\{x: x \in A \wedge x \notin B\}$

Si $C = A \setminus B$ se dice que C es el complemento de B respecto de A

Si $A \subset N$, $N \setminus A = \overset{\text{not.}}{\overline{A}} = A^c$ (complementario de A)

$A \subseteq B: \Leftrightarrow \forall x \in A, x \in B$

$A \subset B: \Leftrightarrow A \subseteq B \wedge \exists x \in B \wedge x \notin A$

$A \times B = \{(a, b): a \in A \wedge b \in B\}$

$A^2 = A \times A$

$A^n = A \overset{(n)}{\times} \dots \times A$



1. Teoría de Conjuntos

- Definición 1.2: Aplicación

Sean A, B conjuntos cualesquiera, se dice que f es una aplicación de A en B si

$$\left. \begin{array}{l} f : A \rightarrow B \\ x \rightarrow f(x) \end{array} \right\| \Leftrightarrow \{(x, f(x))\} \subset A \times B, \forall a \in A \exists ! b \in B / f(a) = b$$

$A \equiv$ Origen de f

$f(A) = C \subseteq B \equiv$ Imagen de f

$f : A \rightarrow B$ es inyectiva $\Leftrightarrow (\forall a, b \in A, a \neq b \Rightarrow f(a) \neq f(b))$

$f : A \rightarrow B$ es sobreyectiva (o suprayectiva) $\Leftrightarrow \forall b \in B \exists a \in A, f(a) = b$

f es biyectiva si es inyectiva y sobreyectiva



1. Teoría de Conjuntos

- Definición 1.3

- 1.- Sea X un conjunto cualquiera, se define el conjunto **potencia de X** , y se denota por $P(X)$, al conjunto formado por todos los subconjuntos de X .
- 2.- Se define como **cardinal de X** , y se denota por $|X|$ al número de elementos de X , si X es finito.

- Teorema 1.1

Sea X un conjunto finito con tamaño mayor o igual que uno, entonces

$$|P(X)| = 2^{|X|}$$



1. Teoría de Conjuntos

- Demostración

Observación: podemos centrar nuestro estudio en conjuntos de números naturales sin perder generalidad, ya que podemos interpretar que estos conjuntos codifican a todos los conjuntos finitos.

$$n=1$$

$$|X| = 1; \quad X = \{1\} \quad \wp(X) = \{\{\}, \{1\}\}$$

$$|\wp(X)| = 2 = 2^1 = 2^{|X|}$$

$$n \Rightarrow n+1$$

$|X|=n \Rightarrow |\wp(X)| = 2^{|X|}$ por hipótesis de inducción

Sea $Y = X \cup \{y\}$ $y \notin X \Rightarrow |Y| = n+1$

Sean $Y_1 = \{A : A \subseteq Y \wedge y \notin A\} = \wp(X)$

$Y_2 = \{B : B \subseteq Y \wedge y \in B\}$

Es obvio que

a) $Y_1 \cap Y_2 = \emptyset$

b) $Y_1, Y_2 \subseteq \wp(Y)$

c) $Y_1 \cup Y_2 = \wp(Y)$



1. Teoría de Conjuntos

- Lema: $|Y_2| = |\wp(X)| = 2^n$

Demostración del lema

Defino la función

$$f: Y_1 \rightarrow Y_2$$

$$A \rightarrow f(A) = A \cup \{y\}$$

Basta con demostrar que f es biyectiva.

Sean $A, B \in Y_1$, $A \neq B \Rightarrow A \cup \{y\} \neq B \cup \{y\} \Rightarrow f(A) \neq f(B) \Rightarrow f$
inyectiva

Sea $P \in Y_2$ entonces $\exists A \in Y_1 / P = A \cup \{y\} \Rightarrow f(A) = P$

- $|Y| = n+1$
- Como se cumple $Y_1 \cap Y_2 = \emptyset$; $Y_1, Y_2 \subseteq \wp(Y)$; $Y_1 \cup Y_2 = \wp(Y)$
 $|\wp(Y)| = |Y_1| + |Y_2| = 2^n + 2^n = 2 \times 2^n = 2^{n+1}$ •



1. Teoría de Conjuntos

- Corolario 1.1: $\forall X$ conjunto finito $|\wp(X)| > |X|$
- Definición 1.4

El **número cardinal** representa al tamaño de un conjunto y que coincide con el número de elementos de un conjunto cuando éste es finito.

- Concepto (que no número) del infinito: ser mayor que cualquier número.
 - Esta característica tendrá consecuencias importantes.
 - Comparar dos números cardinales infinitos.
 - Relación *menor que* entre dos conjuntos cualquiera.



1. Teoría de Conjuntos

- Definición 1.5: Sean X, Y dos conjuntos cualesquiera.
 - i) Se dice que la cardinalidad del conjunto X es menor o igual que la de Y , y se representa con $|X| \leq |Y|$, si y solo si existe una función inyectiva de X en Y .
 - ii) Se dice que la cardinalidad del conjunto X es igual que la de Y , y se representa con $|X| = |Y|$, si y solo si existe una función biyectiva de X en Y .



1. Teoría de Conjuntos

- Teorema 1.2 (Schröder-Berstein): Sean X, Y conjuntos cualesquiera.

Si $|X| \leq |Y|$ y

$$|Y| \leq |X|$$

entonces

$$|X| = |Y|$$

- Observación: $|\mathbb{N}| = |\mathbb{N}^+|$

ya que la función que asocia a cada x de $\mathbb{N}$ con el número $x+1$ de $\mathbb{N}^+$ es biyectiva.

(ejemplo: el hotel de Hilbert)



1. Teoría de Conjuntos

- Teorema 1.3: Si X es un conjunto cualquiera, entonces $|X| < |\wp(X)|$

- Demostración:

Obviamente $|X| \leq |\wp(X)|$ ya que la función f definida

$$\begin{aligned} f: X &\rightarrow \wp(X) \\ x &\rightarrow f(x) = \{x\} \end{aligned}$$

es claramente inyectiva aun en el caso en que X sea vacío.

Falta por probar entonces que $|X| \neq |\wp(X)|$



1. Teoría de Conjuntos

Para probarlo consideremos el conjunto

$$Y = \{x : x \in X \wedge x \notin f(x)\} \subseteq X \Rightarrow Y \in \wp(X)$$

Vamos a demostrar que no existe ninguna función f biyectiva por reducción al absurdo.

Supongamos que f es biyectiva. Entonces

$$f: X \rightarrow \wp(X) \text{ biyectiva} \Rightarrow \exists y \in X / f(y) = Y$$

Ahora bien, ese elemento debe cumplir una de estas dos propiedades:

a) $y \in Y$

b) $y \notin Y$

pero ambas posibilidades conducen a una contradicción.

a) $y \in Y \Rightarrow y \in f(y) = Y \Rightarrow y \notin Y$. Abs.

b) $y \notin Y = f(y) \Rightarrow y \notin f(y) \Rightarrow y \in Y$. Abs.

$$\therefore f \text{ no puede ser biyectiva} \Rightarrow |X| < |\wp(X)| \quad \bullet$$



1. Teoría de Conjuntos

- Corolario 1.2: No existe ningún conjunto cuyo cardinal sea el mayor posible, o dicho de otra forma, no existe un número cardinal máximo.

Demostración

- Sea X un conjunto cualquiera $\Rightarrow |\wp(X)| > |X| \Rightarrow |\wp(\wp(X))| > |\wp(X)| \dots$

¿existirá un número cardinal infinito menor?

- **Axioma de elección:** dada una colección cualquiera de conjuntos no vacíos, existe una función g cuyo dominio es la colección de conjuntos, y que para cada conjunto X de la colección, $g(X) \in X$.

Para un conjunto X de la colección, $g(X) = x \in X$



1. Teoría de Conjuntos

- Teorema 1.4: La cardinalidad del conjunto de los números naturales es menor o igual que la cardinalidad de cualquier conjunto X infinito.

Demostración

Sea X un conjunto infinito.

¿ Existe $f: \mathbb{N} \rightarrow X$ inyectiva ?

Axioma de elección: Sea g una función con dominio en la colección de subconjuntos no vacíos de X y que para cada subconjunto no vacío Y de X , $g(Y)$ es un elemento de Y .



1. Teoría de Conjuntos

Por inducción:

a) $f(0) = g(X)$

b) Sea n un número natural tal que ya hayamos definido $f(n)$, $f(n-1)$, ..., $f(0)$.

Definimos

$$f(n+1) = g(X \setminus \{f(n), f(n-1), \dots, f(0)\}).$$

Nótese que

$X \setminus \{f(n), f(n-1), \dots, f(0)\}$ es un subconjunto no vacío ya que X es infinito y

$\{f(n), f(n-1), \dots, f(0)\}$ es finito, por lo que

$f(n+1)$ debe existir y

ser distinto de $f(n), f(n-1), \dots, f(0) \Rightarrow f$ es inyectiva.



1. Teoría de Conjuntos

- Definición 1.6: **Conjuntos numerables y no numerables**

1.- Decimos que un conjunto cualquiera X es **numerable** si y sólo si cumple una de estas dos propiedades:

a) X es finito

b) X es infinito y $\exists f: X \rightarrow \mathbb{N}$ inyectiva

(Nótese que en virtud del teorema anterior y el de Schröder-Berstein no sería más restrictivo decir que f fuera biyectiva)

2.- Decimos que un conjunto cualquiera X es **no numerable** si no cumple ninguna de las dos propiedades del punto anterior.



1. Teoría de Conjuntos

- Ejemplo: El conjunto $\mathbb{N} \times \mathbb{N}$ es numerable.

Demostración

1. Hay que establecer un método efectivo de numeración para los elementos del conjunto.
2. Demostrar que todos los elementos son contados en algún orden por el método.
3. Encontrar la función f que numera $\mathbb{N} \times \mathbb{N}$ (trivial): función que asigna a cada elemento de la lista el número que representa su posición en la lista



1. Teoría de Conjuntos

- **¡OJO!** Con este método hay que tener mucho cuidado en la construcción de la lista.
- Hay que asegurar que todos y cada uno de los elementos del conjunto aparecen en la lista.
- Intento erróneo de construir la lista:
 - 1.- Incluir en la lista todos los pares de la forma $(1,n)$ donde $n \in \mathbb{N}$
 - 2.- Incluir en la lista los elementos de la forma $(2,n)$ donde $n \in \mathbb{N}$

.....

Error: **nunca dejamos de numerar los elementos de la forma $(1,n)$** , número de pares infinito $\rightarrow$ los elementos de la forma $(2,n)$ no aparecen nunca en la lista.



1. Teoría de Conjuntos

En el caso de nuestro ejemplo, tratamos de construir una lista de pares ordenada:

- Primero, incluimos los pares cuyos elementos suman cero (solo hay uno, el $(0, 0)$), luego los pares que suman 1 ordenados entre ellos según el orden natural de su primera componente:

$(0,1), (1,0)$

- Pares cuyos componentes suman dos:

$(0,2), (1,1), (2,0)$

- Pares que suman tres:

$(0,3), (1,2), (2,1), (3,0);$

- Cualquier par (m,n) : lugar $(m+1)$ -ésimo de los pares que suman $m+n$
- En esta numeración, aparecen todos los elementos de $\mathbb{N} \times \mathbb{N}$
- Concluimos que **el conjunto $\mathbb{N} \times \mathbb{N}$ es numerable**



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

Definición 1.7: Sean A y B dos conjuntos cualesquiera.

Una relación binaria, R , de A en B asigna a cada pareja ordenada $(a,b) \in A \times B$ exactamente uno de estos dos predicados:

- a) “ a está relacionado con b ”, escrito aRb
- b) “ a no está relacionado con b ”

Una relación de un conjunto A en el mismo conjunto A se llama *relación en A* .



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Propiedades de las relaciones: Decimos que una relación R en un conjunto S es
 - a) *reflexiva* si: $\forall a \in S, aRa$
 - b) *irreflexiva* si aRa es falso $\forall a \in S$
 - c) *transitiva* si: $\forall a, b, c \in S / aRb \wedge bRc \Rightarrow aRc$
 - d) *simétrica* si: $\forall a, b \in S / aRb \Rightarrow bRa$
 - e) *antisimétrica* si: $\forall a, b \in S / aRb \Rightarrow bRa$ es falso

(Nótese que una relación antisimétrica debe ser irreflexiva)



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Ejemplos de relaciones:

a) La inclusión de conjuntos es una relación en cualquier clase de conjuntos, ya que dados dos conjuntos A y B ó $A \subset B$ ó $A \not\subset B$.

b) El matrimonio es una relación entre el conjunto H de hombres y el conjunto M de mujeres, ya que dados dos elementos $h \in H$ y $m \in M$, ó h está casado con m o h no está casado con m .

c) La relación $<$ en el conjunto de los enteros es transitiva ya que dados a, b, c enteros, si $a < b$ y $b < c$, entonces $a < c$

d) La relación ser perpendicular en el conjunto de las rectas del plano es irreflexiva y simétrica



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Una relación binaria R define dos conjuntos A y B , que son

$$\text{Dom}(R) = \{ a \in A / \exists x \in B , (a,x) \in R \}$$

(Dominio de R)

$$\text{Im}(R) = \{ b \in B / \exists y \in A , (y,b) \in R \}$$

(Imagen de R)



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Definición 1.8: Si $R \subseteq A \times B$ es una relación entre A y B , entonces

$$R^{-1} = \{(b, a) / (a, b) \in R\} \subseteq B \times A$$

es una relación de B en A . A R^{-1} la llamaremos **relación inversa** de R

- Definición 1.9: Sea A un conjunto no vacío. Una colección $\mathcal{A}$ de subconjuntos no vacíos de A es una partición de A si cumple:

$$\text{i) } B, C \subseteq \mathcal{A} \Rightarrow B = C \vee B \cap C = \emptyset$$

$$\text{ii) } \bigcup_{B \in \mathcal{A}} B = A$$



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Ejemplos: Sea $\mathbb{Q}$ el conjunto de los números racionales y sea $r \in \mathbb{Q}$

1) $Q_r = \{ (x,y) \in \mathbb{Z} \times \mathbb{N}^+ \mid (x/y)=r \}$

Por ejemplo,

$$Q_{3/8} = \{ (3,8), (6,16) \dots \}$$

La colección

$F = \{ Q_r / r \in \mathbb{Q} \}$ es una partición de $\mathbb{N} \times \mathbb{Z}^+$

La demostración se deja como ejercicio...

2) Sea $m \in \mathbb{Z}^+$

$$\mathbb{Z}_i = \{ x \in \mathbb{Z} \mid \exists k \in \mathbb{Z}, x-i = km \}$$

Dado m , la colección

$$\{ \mathbb{Z}_0, \mathbb{Z}_1, \dots, \mathbb{Z}_{m-1} \}$$

es una partición de $\mathbb{Z}$



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Definición 1.10: Sean A, B , y C conjuntos cualesquiera. Sean $R \subseteq A \times B$ y $S \subseteq B \times C$ relaciones. Definimos la **composición de R con S** , y lo notaremos como

$$S \circ R = \{(a, c) \in A \times C / \exists b \in B, (a, b) \in R \wedge (b, c) \in S\}$$

- Observación: La composición de relaciones no es, en general conmutativa
- Ejemplo: Sean $A=B=C=\mathbb{N}$ y sean $R=\{(n, 2n) / n \in \mathbb{N}\}$ y $S=\{(n, n+1) / n \in \mathbb{N}\}$. Entonces se tiene que:

$$S \circ R = \{(n, 2n+1) / n \in \mathbb{N}\}$$

$$R \circ S = \{(n, 2(n+1)) / n \in \mathbb{N}\}$$

claramente $S \circ R \neq R \circ S$



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Definición 1.11: Una relación R en un conjunto S se llama **relación de equivalencia** si es una relación reflexiva, simétrica y transitiva.

Las relaciones de equivalencia generan una colección de subconjuntos disjuntos que “cubren” todo el conjunto S , es decir, crean una **partición del conjunto S** en subconjuntos, llamados **clases de equivalencia**. Es decir, $S = S_1 \cup S_2 \cup \dots$, donde para cada i y j , con $i \neq j$:

- 1) $S_i \cap S_j = \emptyset$;
- 2) Para cada a y b de S_i , aRb ;
- 3) $\forall a \in S_i$ y $b \in S_j$, aRb es falso.

(Nótese que las clases de equivalencia pueden ser infinitas)



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Teorema 1.5: Las clases de equivalencia forman una partición

Demostración:

- i) ¿Disjuntos?

Supongamos que no. Es decir, supongamos que

$$\exists z \in [x] \cap [y] \Rightarrow (x,z), (y,z) \in R \Rightarrow$$

$$(z,y) \in R \text{ (por ser } R \text{ simétrica)} \Rightarrow$$

$$(x,y) \in R \text{ (por ser } R \text{ transitiva)} \Rightarrow x \in [y]$$

por un razonamiento análogo

$$y \in [x]$$

Ademas, si $t \in [x] \Rightarrow (x,t) \in R \Rightarrow (t,y) \in R \Rightarrow t \in [y] \Rightarrow$

$$[x] \subseteq [y]$$

Y de nuevo análogamente, $[y] \subseteq [x]$, de lo que se concluye

$$[x] = [y] .$$



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

$$\text{ii)} \quad \bigcup_{x \in S} [x] = S \quad ?$$

$$\forall x \in S \ (x, x) \in R \ (\text{por ser } R \text{ reflexiva}) \Rightarrow \\ x \in [x] \ \forall x \in S \cdot$$



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Teorema 1.6: Cualquier partición A de un conjunto no vacío X , define una relación de equivalencia.
- Notación:
 $aRb \Leftrightarrow (a,b) \in R$
 $R = \{(a,b) / aRb \}$
 $a \in S, [a] \equiv$ la clase de equivalencia en la que se encuentra “ a ”
- Ejemplos
 - a) La clasificación de animales por especies, es decir, la relación “ser de la misma especie que” es una relación de equivalencia.
 - b) La relación de inclusión entre conjuntos no es una relación de equivalencia, ya que es reflexiva y transitiva pero no simétrica ($A \subseteq B$ no implica $B \subseteq A$)
 - c) Sea m un entero positivo. Dos enteros a y b se dice que son congruentes módulo m , y se escribe $a \equiv b \pmod{m}$, si m divide a $a-b$. Por ejemplo, para $m = 4$ tenemos que $11 \equiv 3 \pmod{4}$ ya que 4 divide a $11-3$. Pues bien, la relación de congruencia módulo m es una relación de equivalencia. Además, existen exactamente m clases de equivalencia.
- Demostración teorema 1.6: (Ejercicio)



1. Teoría de conjuntos.

Relaciones. Relaciones de equivalencia. Conjuntos cocientes.

- Definición 1.12: Sea S un conjunto y R una relación de equivalencia en R . Al conjunto formado por todas las clases de equivalencia de elementos de S se le denomina **conjunto cociente de S por R** , y se escribe como S/R .
- Definición 1.13: Sea $\wp$ un conjunto de propiedades de relación sobre conjuntos. Definimos la $\wp$ -clausura de una relación R como la menor relación R' que incluye todos los pares de R y cumple las propiedades de $\wp$.
- Ejemplos: La clausura transitiva de R , denotada por R^+ , se define como:
 - a) Si $(a,b) \in R \Rightarrow (a,b) \in R^+$
 - b) Si $(a,b) \in R^+ \wedge (b,c) \in R \Rightarrow (a,c) \in R^+$
 - c) Nada está en R^+ que no cumpla los puntos anteriores.La clausura transitiva y reflexiva de R , denotada por R^* , es

$$R^* = R^+ \cup \{(a,a) / a \in S\}$$



1. Teoría de conjuntos.

Funciones.

Definición 1.14: Una función de A en B es un subconjunto de $A \times B$ en el que no existen dos pares con el mismo primer componente, o lo que es lo mismo, f es una función si

$$f = \{(x,y) \subseteq A \times B \mid (x,y), (x,z) \in f \Rightarrow y = z\}$$

Definición 1.15: Sea f una función de A en B . Definimos el *Dominio de f* , y lo denotaremos como $\text{Dom}(f)$, como el subconjunto de A para el que f está definida, es decir:

$$\text{Dom}(f) = \{x \in A \mid \exists y \in B \wedge (x,y) \in f\}$$



1. Teoría de conjuntos.

Funciones.

- Definición 1.16:

- i) Función parcial

Una función f de A en B se dice que es una función parcial si:

$$\text{Dom}(f) \subseteq A$$

Diremos que f es estrictamente parcial si:

$$\text{Dom}(f) \subset A$$

- ii) Función total

Una función f de A en B se dice que es una función total si:

$$\text{Dom}(f) = A$$

Notación: escribiremos $f: A \rightarrow B$ para representar que f es una función total o cuando la totalidad o parcialidad de f carezca de importancia. Escribiremos $f: A \dashrightarrow B$ para poner de manifiesto que f es una función parcial.



1. Teoría de conjuntos.

Funciones.

- Teorema 1.7: Sean f y g funciones totales de A en B .
Entonces

$$f = g \Leftrightarrow f(x) = g(x) \forall x \in A$$

Demostración:

$\Rightarrow$

$$x \in A \text{ y } y = f(x) \Rightarrow (x, y) \in f \Rightarrow (x, y) \in g \Rightarrow y = g(x)$$

$\Leftarrow$

$\subseteq$

$$\begin{aligned} \text{Sea } (x, y) \in f &\Rightarrow y = f(x) \Rightarrow y = g(x) \Rightarrow \\ (x, y) &\in g \Rightarrow f \subseteq g \end{aligned}$$

$\supseteq$

Análogo .



1. Teoría de conjuntos.

Funciones.

- Definición 1.17: (Imagen inversa)

Sea $f: A \rightarrow B$ y sea $Y \subseteq B$. Se define la *imagen inversa de Y bajo f* , y se nota como $f^{-1}(Y)$, como:

$$f^{-1}(Y) = \{x \in A / \exists y \in Y, f(x) = y\}$$

- Observación: en general, f^{-1} no tiene por que ser una función. ¿Cuándo será? (Ejercicio)



1. Teoría de conjuntos.

Funciones.

Teorema 1.8: Sea $f: A \rightarrow B$. Entonces:

1.- $f(\emptyset) = \emptyset$

2.- $f(\{x\}) = \{f(x)\}$

3.- $X \subseteq Y \subseteq A \Rightarrow f(X) \subseteq f(Y)$

4.- Si $X \subseteq Y \subseteq B \Rightarrow f^{-1}(X) \subseteq f^{-1}(Y)$

5.- Si $X, Y \subseteq B \Rightarrow f^{-1}(X \setminus Y) = f^{-1}(X) \setminus f^{-1}(Y)$

Demostración: (ejercicio)



1. Teoría de conjuntos.

Funciones.

Definición 1.18: Sean A, B, C conjuntos cualesquiera y sean f y g funciones como sigue:

$$f: A \rightarrow B$$

$$g: B \rightarrow C$$

Se define la **composición de f con g** , y se nota como $g \circ f$, como la función:

$$g \circ f = \{(a, c) \in A \times C \mid \exists b \in B, f(a) = b \wedge g(b) = c\}$$

o lo que es lo mismo:

$$g \circ f = \{(a, g(f(a)))\} \subseteq A \times C$$

Observación: si f o g son funciones parciales, la composición deberá restringirse a los dominios correspondientes.



1. Teoría de conjuntos.

Funciones.

Ejemplo:

Sean

$$f: \mathbb{N} \rightarrow \mathbb{N}$$

$$g: \mathbb{N} \rightarrow \mathbb{N}$$

tal que

$$f(x) = 2x \quad \forall x \in \mathbb{N}$$

$$g(x) = x+1 \quad \forall x \in \mathbb{N}$$

entonces, se tiene que:

$$g \circ f: \mathbb{N} \rightarrow \mathbb{N}$$

$$g \circ f(x) = 2x + 1 \quad \forall x \in \mathbb{N}$$

$$f \circ g: \mathbb{N} \rightarrow \mathbb{N}$$

$$f \circ g(x) = 2(x+1)$$



1. Teoría de conjuntos.

Isomorfismos y homomorfismos.

- Definición 1.19: Sean $f: A \rightarrow A$ y $g: B \rightarrow B$ dos funciones sobre A y B respectivamente. Si existe una función t que transforma todos los x de A en algún $u=t(x)$ de B tal que $g(t(x)) = t(f(x))$, entonces se dice que f y g son *homomorfas* y a t se le llama *homomorfismo*.

Si además t es biyectiva, entonces f y g se denominan *isomorfas* y a t , *isomorfismo*.

Si h es un isomorfismo, entonces tiene inversa. Es decir, todos los isomorfismos son reversibles. Pero no ocurre, en general, lo mismo con los homomorfismos.



1. Teoría de conjuntos.

Isomorfismos y homomorfismos.

- Definición 1.20:

Todo isomorfismo $t: A \rightarrow A$ se denomina *automorfismo*.

Todo homomorfismo $t: A \rightarrow A$ se denomina *endomorfismo*.

Se dice que dos funciones f y g son automorfismos/endomorfismos si existe un automorfismo/endomorfismo entre ellas.



2. Funciones algorítmicas

- Programas: funciones entre números naturales codificados en binario
- Dado un alfabeto Σ , las palabras de Σ^* representan posibles datos de entrada para algoritmos cuyo alfabeto asociado sea Σ
- Cómputo:
 - **a**: algoritmo
 - n-tupla $x^n(x_0, x_1, x_2, \dots, x_{n-1})$: n datos de entrada
 - Un dato de salida
 - Ejecución finaliza:
 - Número finito de pasos: $\mathbf{a}(x_0, x_1, x_2, \dots, x_{n-1}) \in \Sigma^*$
 - Número infinito de pasos: $\mathbf{a}(x_0, x_1, x_2, \dots, x_{n-1}) = \infty$

Ejemplo: cálculo de la raíz cuadrada cuando $x=2$

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$



2. Funciones algorítmicas

- Todo algoritmo $\mathbf{a}$ define una función $\varphi_{\mathbf{a}}^n$:
$$\text{dom} (\varphi_{\mathbf{a}}^n) := \{x^n / x^n \in (\Sigma^*)^n, \mathbf{a}(x^n) \neq \infty\}$$

Para $x^n \in \text{dom} (\varphi_{\mathbf{a}}^n)$: $\varphi_{\mathbf{a}}^n (x^n) = \mathbf{a}(x^n)$.
- Una función ψ que tenga como dominio un subconjunto de $(\Sigma^*)^n$ y tome valores en Σ^* se le llama **función parcial n-aria sobre Σ^*** .
- Si el dominio de ψ es todo $(\Sigma^*)^n$, se dice que ψ es **total**.



2. Funciones algorítmicas

- Definición 1.33: Sea ψ una función parcial n -aria sobre Σ^* . Se dice que ψ es **algorítmica** si existe un algoritmo a de alfabeto asociado Σ tal que

$$\varphi_a^n = \psi \text{ (se dice que } a \text{ computa } \psi).$$

- Si además a nos ha sido dado de manera efectiva, se dice que ψ es **efectivamente algorítmica**.



2. Funciones algorítmicas

- Ejemplo: El conjunto $\mathbb{N}$ puede representarse, con

$$\Sigma = \{ | \}^*$$

$$\lambda=0, |=1, ||=2, |||=3, \dots$$

La función total binaria ψ definida por

$$\psi(x,y) = \text{m.c.d.}(x,y)$$

es **algorítmica**, y **efectivamente algorítmica** ya que podemos dar un algoritmo concreto que la calcula.



2. Funciones algorítmicas

Por ejemplo, el siguiente algoritmo calcula el m.c.d. de dos números:

- 1) Tomar x como valor de a e y como valor de b
- 2) Hacer la división entera de a entre b ; sea r el resto de esta división
- 3) Si $r=0$, tomar b como valor de z y detener el proceso;
 en caso contrario, tomar b como nuevo valor de a y r como nuevo valor de b , y pasar a la instrucción 2.

El resultado se guarda en z .



2. Funciones algorítmicas

- Toda función efectivamente algorítmica es, evidentemente, algorítmica.
- El recíproco no es cierto: hay funciones algorítmicas que no son (hoy por hoy) efectivamente algorítmicas.

Ejemplo: la función total $\psi: \mathbb{N} \rightarrow \mathbb{N}$ definida por:

$$\psi(x) := \begin{cases} 0 & \text{si cuando expiró el último dinosaurio era de día} \\ 1 & \text{en caso contrario} \end{cases}$$

es algorítmica;

ψ no es efectivamente algorítmica.



2. Funciones algorítmicas

Así pues, hemos clasificado las funciones en tres categorías:

- efectivamente algorítmicas
- algorítmicas
- no algorítmicas

Pregunta: ¿Existen funciones no algorítmicas?



2. Funciones algorítmicas

Teorema 1.10: Existen funciones no algorítmicas

Demostración (ejercicio):

Pistas:

1. Sea T el conjunto de todas las funciones de $\mathbb{N}$ en $\mathbb{N}$. Calcular su cardinal.
 2. Sea A el conjunto de todas las funciones algorítmicas. Calcular su cardinal.
-
1. Conclusión: hay más funciones en T que en A , luego alguna será no algorítmica



2. Funciones algorítmicas

- Ejemplo: El conjunto $\mathbb{N}$ puede representarse, con

$$\Sigma = \{ | \}^*$$

$$\lambda=0, |=1, ||=2, |||=3, \dots$$

La función total binaria ψ definida por

$$\psi(x,y) = \text{m.c.d.}(x,y)$$

es **algorítmica**, y **efectivamente algorítmica** ya que podemos dar un algoritmo concreto que la calcula.



2. Funciones algorítmicas

Por ejemplo, el siguiente algoritmo calcula el m.c.d. de dos números:

- 1) Tomar x como valor de a e y como valor de b
- 2) Hacer la división entera de a entre b ; sea r el resto de esta división
- 3) Si $r=0$, tomar b como valor de z y detener el proceso;

en caso contrario, tomar b como nuevo valor de a y r como nuevo valor de b , y pasar a la instrucción 2.

El resultado se guarda en z .



2. Funciones algorítmicas

Definición: Todo programa lleva asociada una función (convierte un dato de entrada en un dato de salida). Una función se dice computable si existe un programa que la computa.

Teorema: Una función es computable **si y solo si** es algorítmica

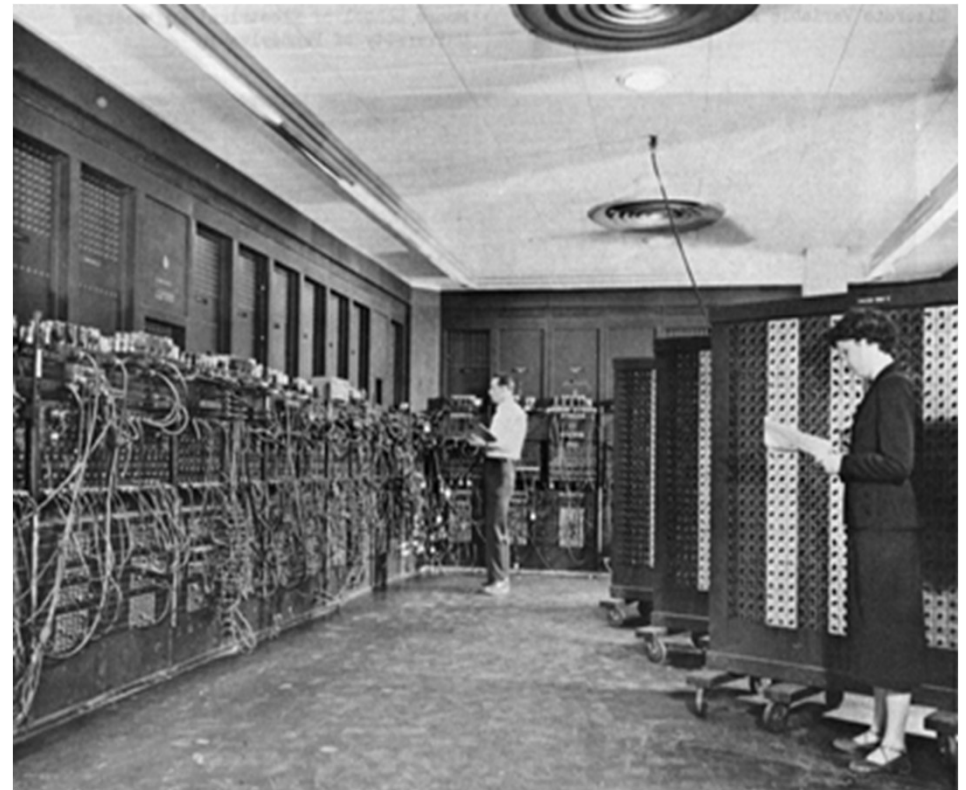
Pistas

1. Un algoritmo es un programa en pseudocódigo $\Rightarrow$ su función algorítmica es computable
2. Un programa se puede escribir en lenguaje fuente (alfabeto finito, texto finito), luego es una representación de un algoritmo $\Rightarrow$ su función computable es algorítmica



3. Computación y compiladores

- Llegada de John Von Neumann al proyecto ENIAC (1945) → cablear el ordenador para cada nueva tarea.
- Decide resolver este problema, consiguiéndolo en 1949, escribiendo secuencias de código o programas que hacen que estos ordenadores realizaran los cálculos deseados.
- Estos programas se escribían utilizando códigos numéricos que representaban las operaciones que se iban a realizar.
- El lenguaje utilizado para construir estos programas se denominó lenguaje máquina porque estaba totalmente relacionado con el hardware de la máquina.



¿Por qué son necesarios los compiladores?

- Como se puede ver en el siguiente ejemplo, esta forma de escribir un programa es difícil y tediosa por lo que pronto se dio nombre a los códigos de las operaciones y a las direcciones de memoria
- Un ejemplo de este código máquina es el siguiente:

	<u>CODIGO OPERACIÓN</u>	<u>DIRECCIÓN</u>
<u>CODIGO MÁQUINA</u>	00010101	10000011
<u>ENSAMBLADOR</u>	LOAD	X

- Con el lenguaje ensamblador se mejoró enormemente la velocidad y exactitud con la que se escribían los programas, de hecho todavía se encuentra en uso en situaciones donde se necesita velocidad y se tiene poco espacio para el código



¿Por qué son necesarios los compiladores?

- **Desventajas del lenguaje ensamblador:**
 - No es fácil de escribir
 - Es difícil de leer y entender
 - Depende de la máquina para la que se ha escrito, lo que implica reescribirlo si se va a ejecutar en otra máquina.



SEGUIMOS UNIDOS A LA MÁQUINA O PROCESADOR



¿Cómo nos independizamos de la máquina?

- Generando un código intermedio, denominado código objeto → ¿qué ganamos? El no tener que recordar las direcciones de memoria, así como otros aspectos totalmente ligados con la máquina. Esto se pensaba que no iba a ser fácil, además de poco eficiente.

	<u>CODIGO OPERACIÓN</u>	<u>DIRECCIÓN</u>
<u>ENSAMBLADOR</u>	MOV X, 5	10000011
<u>CÓDIGO</u>	X = 5	X

- Había que conseguir crear un lenguaje que nos permitiera realizar acciones de tal forma que fuera fácil de manejar y aprender por una persona. Este tipo de lenguajes se denominó de **alto nivel**, en contraposición con los de **bajo nivel** (código máquina y ensamblador) totalmente dependientes de la máquina.



¿Cómo nos independizamos de la máquina?

- De esta forma se llegó al que se denomina el primer compilador
 - Grace Hopper, el A-0. Se empezó a trabajar en él en el otoño de 1951 y la primera rutina “compilada” se probó con éxito en un UNIVAC en la primavera de 1952 (Hopper & Mauchly, 1953).
 - El concepto de compilador de estos momentos consistía en:
 - Aceptación del pseudocódigo
 - Su decodificación
 - La búsqueda de la subrutina apropiada
 - La asignación de las direcciones de memoria a las subrutinas en el programa,
 - La selección e inserción de las posiciones de memoria de los argumentos y resultados
 - La organización de la transferencia del control y la escritura del programa terminado en una cinta.
- Esto era más un **cargador o enlazador** que el concepto actual de compilador, pero fue la primera vez que se usó la palabra **compiler**



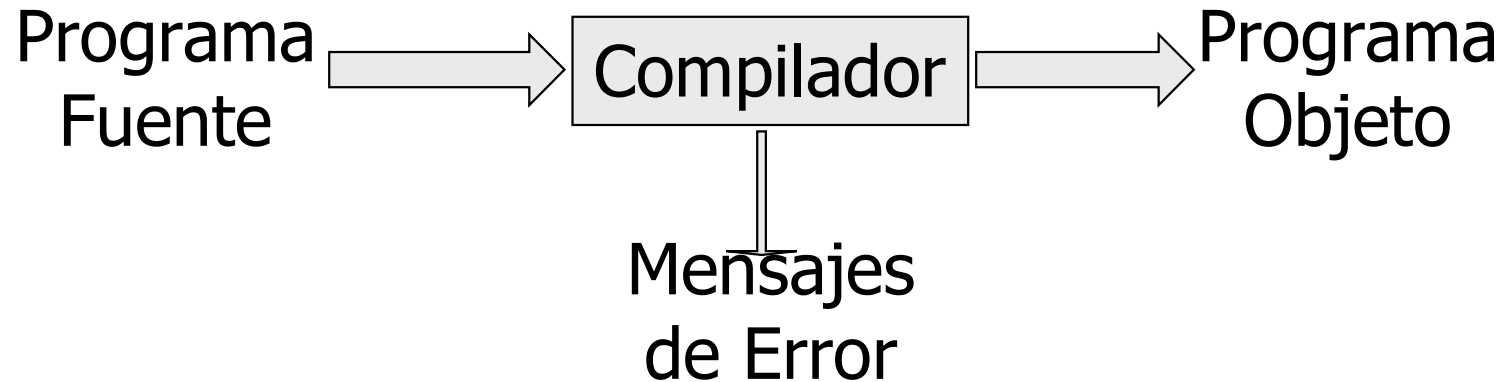
¿Cómo nos independizamos de la máquina?

- El A-0 de Grace Hooper era más un **cargador o enlazador** que el concepto actual de compilador, pero fue la primera vez que se usó la palabra **compiler**.
- Por otro lado, en 1954, **John Backus** comenzó el desarrollo de un compilador de FORTRAN para IBM, concretamente para el IBM 704, que le llevó dos años y medio y 18 hombres para realizarlo. Consistió en dos componentes:
 - el lenguaje FORTRAN y
 - el traductor para el IBM 704.
- Este desarrollo fué un verdadero compilador tal y como lo entendemos actualmente



Compilador. DEFINICIÓN

- Proceso de traducción que convierte un programa fuente escrito en un lenguaje de alto nivel a un programa objeto en código máquina y listo por tanto para ejecutarse en el ordenador, con poca o ninguna preparación adicional



Herramientas necesarias para los Compiladores

- **Los procesadores de lenguaje**, es el nombre genérico que reciben todas las aplicaciones informáticas en las cuales uno de los datos fundamentales de entrada es un **lenguaje. Esto engloba a las herramientas necesarias para un compilador:**
 - Editores (editors)
 - Traductores (translators)
 - Compiladores (compilers)
 - Intérpretes (interpreter)
 - Ensambladores (assemblers)
 - Montadores de enlace o enlazadores (linkers)
 - Cargadores (loaders)
 - Desensambladores (disassemblers)
 - Decompiladores (decompilers)
 - Depuradores (debuggers)
 - Analizadores de rendimiento (profilers)
 - Optimizadores de código (code optimizers)
 - Compresores (compressors)



Herramientas necesarias para los Compiladores

- **Editores:** Son herramientas software que se utilizan para leer y escribir los programas que posteriormente el compilador traducirá a código máquina. Actualmente, se integran con los compiladores (.NET) constituyendo un IDE (entorno de desarrollo interactivo), aunque se pueden utilizar por separado (Eclipse). Para que los ficheros que generan se puedan leer por cualquier editor es necesario que tengan un formato estándar, concretamente ASCII.
- **Traductores:** Como su propio nombre indica, traducen o convierten un programa escrito en un lenguaje (lenguaje fuente) a otro lenguaje (lenguaje objeto). El lenguaje fuente puede ser cualquier lenguaje, normalmente de alto nivel como Visual Basic, Java, COBOL, C++, C#, etc. y el lenguaje objeto puede ser desde otro lenguaje de alto nivel o bien ensamblador o código máquina.
- **Intérpretes:** Es un traductor de lenguaje, pero que a diferencia de un compilador, el código que genera se ejecuta al mismo tiempo que se va traduciendo y no genera como en el caso del compilador un fichero ejecutable. Los lenguajes típicos que utilizan intérprete en lugar de compilador son BASIC y LISP.



Herramientas necesarias para los Compiladores

- **Ensambladores:** Es un traductor para una máquina en particular. Traduce los nombres simbólicos que tienen las instrucciones (Ej: **LOAD**) a cada una de las instrucciones en código binario (**00010101**) que entiende la máquina para la que está diseñado.
- **Preprocesadores:** Es un programa que funciona de forma independiente una vez que el compilador lo llama, antes de comenzar el proceso de compilación. Las tareas que realiza un preprocesador son la sustitución de las macros por las sentencias que las componen, la inclusión de otros archivos o librerías o la eliminación de comentarios.
- **Montadores de enlace:** Puesto que muchos programas se compilan en momentos diferentes, nos hace falta un montador de enlace o enlazador que sea capaz de unir los diferentes módulos con sus respectivos códigos objeto para producir un fichero ejecutable. También sirve para enlazar el código objeto con las funciones que necesita de una librería de funciones u otros módulos, además de con la memoria o los dispositivos de entrada y salida utilizando para ello llamadas a funciones del sistema operativo.



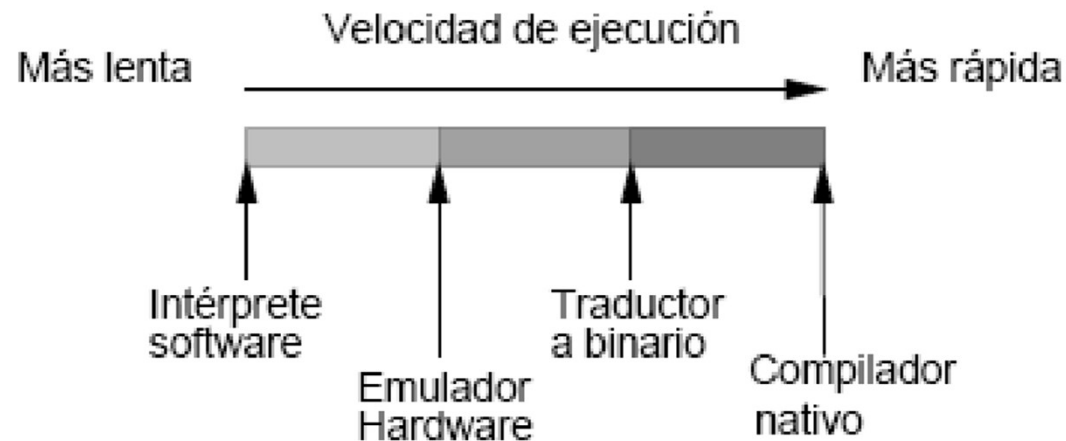
Herramientas necesarias para los Compiladores

- **Cargadores:** Esta herramienta software se encarga asignar las direcciones y el espacio de memoria necesario para la ejecución del programa. Esto se puede realizar porque el montador de enlace le proporciona direcciones de memoria relativas, denominándose este tipo de código reubicable o relocizable, y el cargador le asigna posiciones reales o absolutas. El cargador es normalmente una herramienta que trae el sistema operativo, en el caso de Linux se denomina ld (/lib/ld-linux.so) y en el del antiguo MS-DOS, el cargador se encontraba dentro del intérprete de comandos COMMAND.COM (Cueva, 1998).
- **Depuradores:** Son herramientas que nos permiten encontrar los errores y solucionarlos en un programa, una vez éste ha sido compilado. Normalmente permiten seleccionar variables y visualizar los valores que van teniendo a partir de una ejecución paso a paso
- **Desensambladores:** Es una herramienta que traduce el lenguaje máquina a lenguaje ensamblador. Se utiliza para realizar ingeniería inversa.



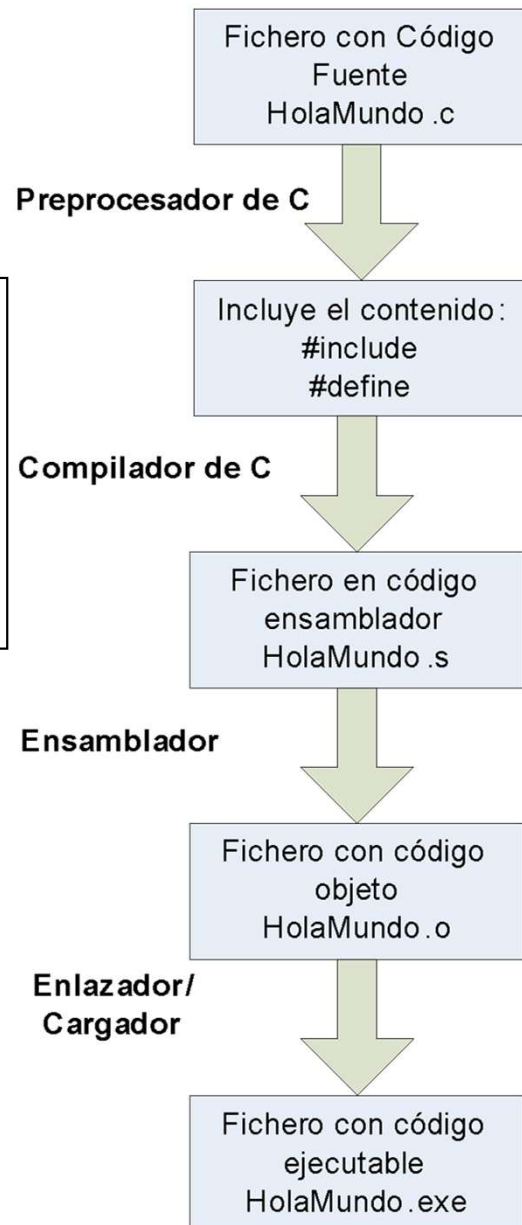
Herramientas necesarias para los Compiladores

- Solución:



- Intérpretes software o emuladores software:
 - Es un programa de código binario que lee una a una las instrucciones binarias de la arquitectura antigua que están en un fichero ejecutable y las interpreta.
 - Ej: Los intérpretes de la máquina abstracta JVM y que permite ejecutar los códigos binarios (bytecodes → .class)

Proceso de Compilación



El **preprocesador** de C (cpp) incluye el contenido de un fichero (Ej: #include <stdio.h>) a través de las directivas #include, #define o #if

Para la creación del fichero de código fuente utilizamos un **editor**, como por ejemplo Eclipse, que incorpora un depurador para ayudarnos a encontrar errores

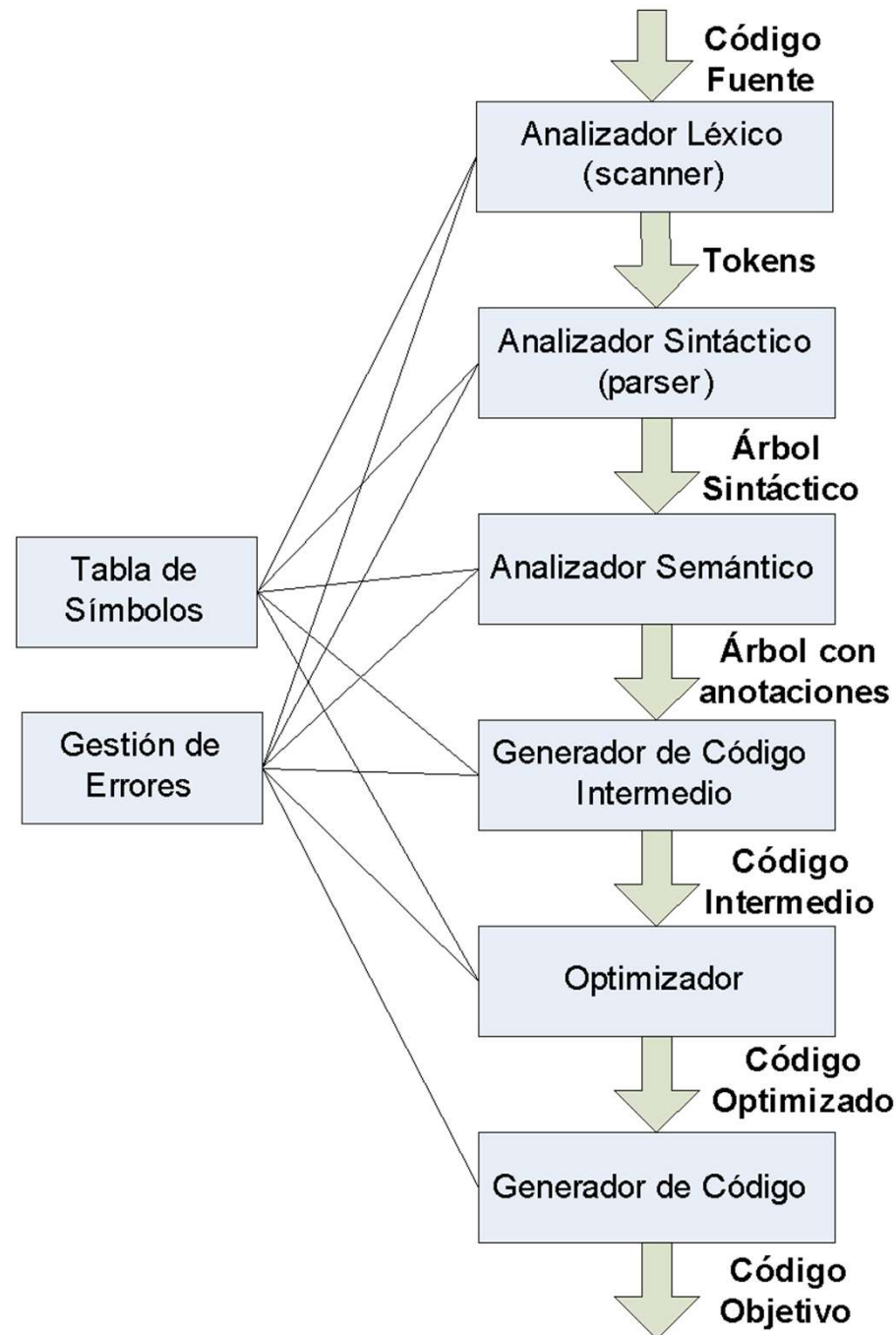
Traduce el código fuente una vez lo ha analizado a código ensamblador. Este proceso lo explicaremos detalladamente en la siguiente pantalla.

Una vez ha procesado el ensamblador del procesador que tenemos en nuestro ordenador, se genera el **código objeto** que ya es código binario.

Se **enlaza** el código objeto con otros ficheros que forman parte de nuestro programa (el enlazador facilita una compilación separada) y **el cargador** le proporciona direcciones de memoria relativas.



Fases de la Compilación



Se comienza a leer la cadena de entrada y los caracteres que se reconocen se agrupan en componentes léxicos (tokens)

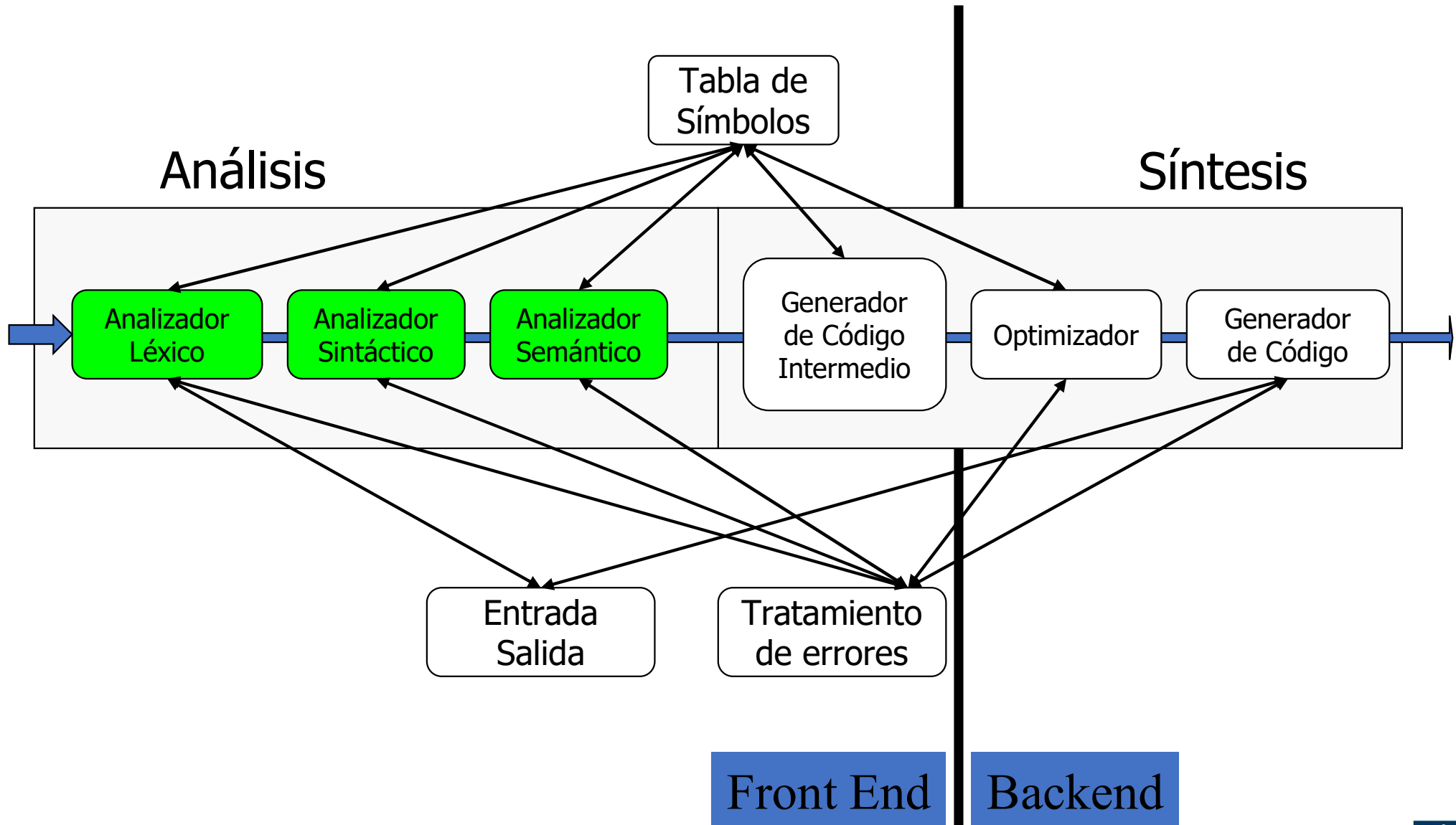
Agrupar los tokens en frases que la gramática del lenguaje reconoce construyendo un árbol sintáctico

El análisis semántico identifica el comportamiento, busca errores semánticos, verifica los tipos, determina operadores y operandos. Para ello utiliza atributos que se añaden al árbol, construyendo así el árbol con anotaciones (o decorado).

Se intenta mejorar el código intermedio, y por tanto el código objetivo mediante distintas técnicas.



Fases de la Compilación



Fases de la Compilación . Análisis del programa fuente

- En la compilación, el análisis consta de 3 fases:
 - Análisis léxico. El analizador léxico (*scanner*) lee la cadena de caracteres del programa fuente, agrupándolos en componentes léxicos (*tokens*).

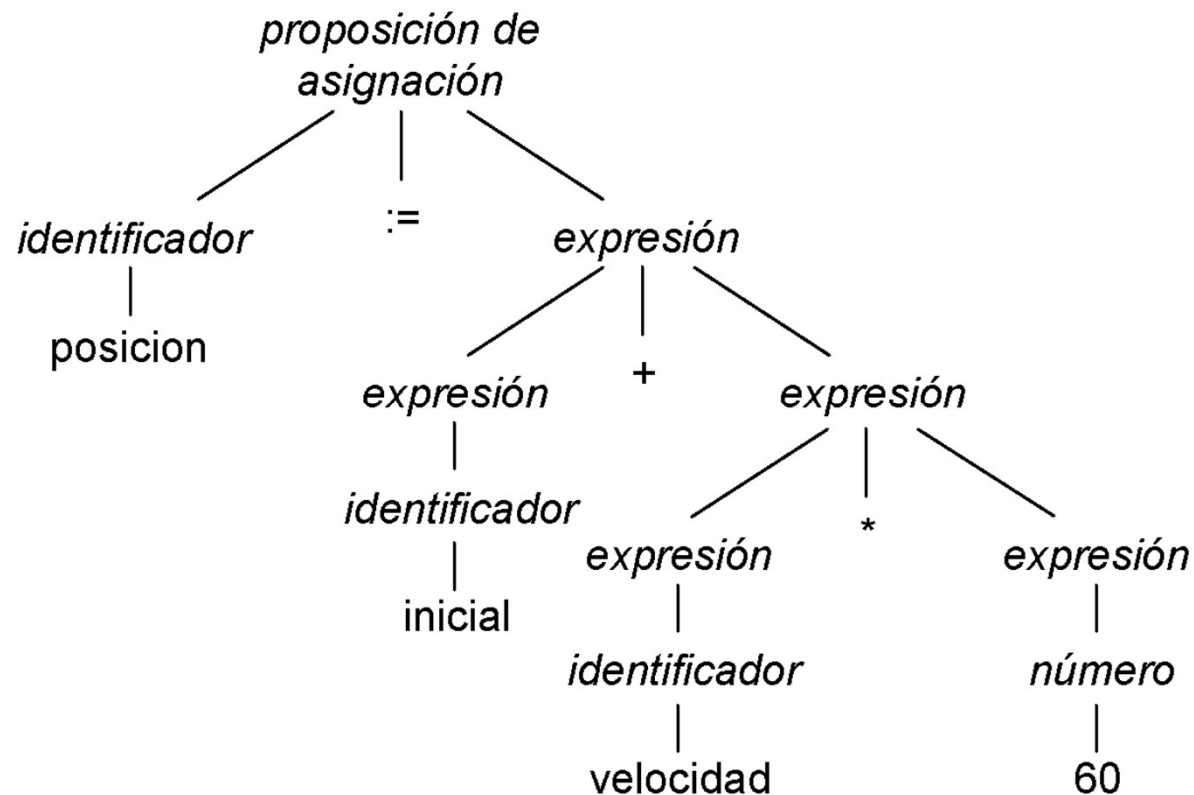
`posicion := inicial + velocidad * 60`

- Identificadores: `posicion`, `inicial`, `velocidad`.
- Operador de asignación: `:=`.
- Operador de suma: `+`.
- Número: `60`.



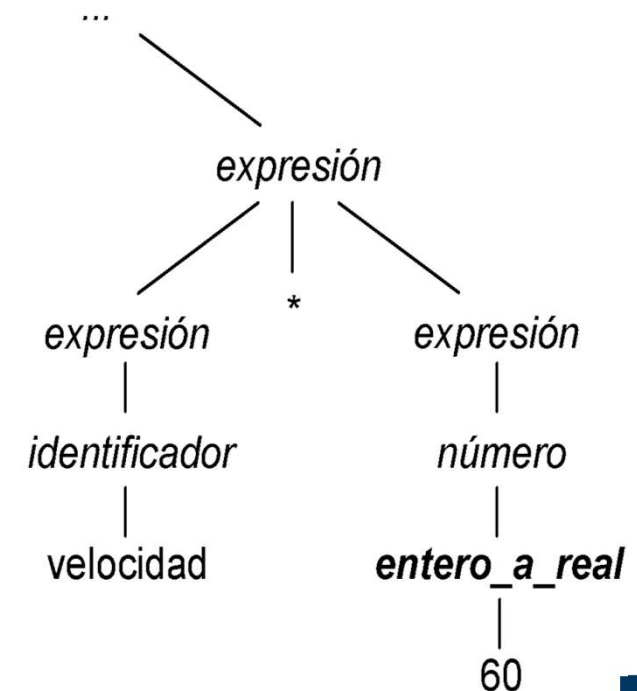
Fases de la Compilación. Análisis del programa fuente

- Análisis sintáctico. El analizador sintáctico agrupa los tokens en frases gramaticales que se usan para sintetizar la salida. Se representa con ayuda de un árbol de análisis sintáctico.



Fases de la Compilación. Análisis del programa fuente

- Análisis semántico. Revisa el programa fuente para comprobar la validez semántica de las sentencias aceptadas por el analizador sintáctico.
 - Un componente importante es la verificación de tipos. El compilador comprueba que cada operador tiene operandos
 - Supongamos que *posición*, *inicial* y *velocidad* están declarados de tipo real, y que 60 es un número entero.
 - El analizador semántico se ocupa de comprobar que *velocidad* y 60 sean del mismo tipo.
 - En este caso, se puede promocionar el tipo de 60 a real, para que no se produzca un error.



Autómatas

Alfabetos, palabras y lenguajes. Lenguajes
Regulares.



Definiciones Básicas

- **Carácter o símbolo:** Es el componente indivisible y elemental con el que se compone un texto.
 - Ejemplos: a, b, 5, [,)
- **Alfabeto Σ :** Es un conjunto de símbolos. Para los lenguajes formales, utilizamos alfabetos que contienen una cantidad finita de símbolos. Ejemplos:
 - $\Sigma_1 = (a, b, c, d, e, f, g, h, i)$.
 - $\Sigma_2 = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)$.
 - $\Sigma_3 =$ El alfabeto del lenguaje de programación C.
- **Palabra w :** Secuencia finita de letras del alfabeto
 - $abc \rightarrow$ es una palabra definida sobre Σ_1
 - $12 \rightarrow$ es una palabra definida sobre Σ_2
 - $main \rightarrow$ es una palabra definida sobre Σ_3



Definiciones Básicas

- **Palabra vacía λ , ϵ :** Es una palabra que no contiene símbolos (es una convención similar al número 0 en matemáticas). Palabra de longitud 0
- **Universo $W(\Sigma^*)$:** Lo constituyen todas las palabras que pueden formarse con símbolos del alfabeto Σ , incluyendo a la palabra vacía (λ). Contiene, por tanto, un número infinito de palabras
 - Supongamos el alfabeto $\Sigma_1 = (a, b, c, d, e, f, g, h, i)$. El universo de ese alfabeto sería: $W(\Sigma_1) = \{\lambda, a, aa, ba, ab, ac, ca, de, \dots\}$
- **Lenguaje L :** Es cualquier subconjunto del universo sobre ese alfabeto Σ , y por tanto, puede ser finito o infinito.
 - Supongamos el alfabeto $\Sigma_1 = (a, b, c, d, e, f, g, h, i)$. Posibles lenguajes de ese alfabeto podrían ser:
 - $L_1 = \{\lambda, aa, bb, cc, dd\}$
 - $L_2 = \{ba, ca, da, fa, ga, ha\}$



Palabras

- Definición: Llamaremos **longitud de una palabra x** sobre un alfabeto Σ al número de elementos de Σ que componen x . Representaremos por $|x|$ a la longitud de la palabra x .
- Obs: la **palabra vacía** es aquella palabra x tal que,
 $|x| = 0$.
- Definición: Al conjunto de todas las palabras de longitud m en Σ lo denotaremos por Σ^m .
- Observación: $\Sigma^0 = \lambda \ \forall \Sigma$ alfabeto, ya que Σ^0 es el conjunto de todas las palabras de longitud cero sobre Σ .



Palabras

- Obs: Sea Σ un alfabeto, se puede definir el **universo de discurso o lenguaje universal**, y se denota por Σ^* , al conjunto de todas las palabras posibles sobre Σ , es decir:

$$\Sigma^* = \bigcup_{m \geq 0} \Sigma^m$$

- Se define también el conjunto Σ^+ como sigue:

$$\Sigma^+ = \bigcup_{m > 0} \Sigma^m$$

- Observacion:

$$\Sigma^+ = \Sigma^* \setminus \{\lambda\}$$

PREGUNTA “de examen”:

¿ $\Sigma^* = \Sigma^+ \cup \{\lambda\}$?

¿Demostración?



Palabras

- Corolario:

- a) λ pertenece a cualquier lenguaje universal.
- b) Σ^* es un conjunto infinito $\forall \Sigma$.

Ejemplo:

Sea $\Sigma = \{A\}$, entonces

$$\Sigma^* = \{\lambda, A, AA, AAA, AAAA, \dots\}$$

Observación: Podemos redefinir la longitud de una palabra x como la función

$$| \cdot | : \Sigma^* \rightarrow \mathbb{N}$$

tal que

$$|\lambda| = 0$$

$$|\omega a| = |\omega| + 1, \forall \omega \in \Sigma^* \wedge a \in \Sigma$$



Operaciones con Palabras

- **CONCATENACIÓN, POTENCIA Y REFLEXIÓN**
- **Concatenación:** Dadas dos palabras, α y β , de un mismo alfabeto Σ , la concatenación de estas dos palabras es una nueva palabra formada por los símbolos de α , seguido de los símbolos de β . Esta operación se representa mediante un punto $\alpha.\beta$, que suele omitirse.
- Ejemplo: Sea $\Sigma_1 = (a, b, c, d, e, f, g, h, i)$, si $\alpha = ba$ y $\beta = ca$, = baca
 - Esta operación no es conmutativa puesto que $\beta.\alpha$, da otra palabra diferente, en este caso sería, $\beta.\alpha = caba$
 - El elemento neutro de esta operación es la palabra vacía (λ).



Operaciones con Palabras

- **Potencia:** Si concatenamos la misma palabra varias veces efectuamos la operación potencia. Esto sería la potencia i -ésima de esa palabra.
 - Ejemplo: Sea $\Sigma_1 = (a, b, c, d, e, f, g, h, i)$, si $\alpha = ba$, $\alpha^2 = \alpha.\alpha = baba$
 - Por definición, cualquier palabra elevada a 0 es la palabra vacía, $\alpha^0 = \lambda$
- **Reflexión:** La palabra inversa de una dada, se construye invirtiendo el orden de los símbolos de la palabra.
 - Ejemplo: Sea $\Sigma_1 = (a, b, c, d, e, f, g, h, i)$, si $\alpha = ba$, $\alpha^{-1} = ab$



Operaciones con Palabras (seamos formales)

Sea $z, x, y \in \Sigma^*$ tal que $z = x \cdot y$. Decimos que x es **cabeza o prefijo** de z y que y es **cola o sufijo** de z . Además, x es **prefijo propio** de z si

$$z = x \cdot y, x \text{ es prefijo propio de } z \Leftrightarrow y \neq \lambda$$

de igual forma, diremos que y es **sufijo propio** de z si

$$z = x \cdot y, y \text{ es sufijo propio de } z \Leftrightarrow x \neq \lambda$$

Ejemplo: Sea $z = ABC$

$x = \lambda, A, AB, ABC$ son palabras prefijos de z

$y = \lambda, C, BC, ABC$ son palabras sufijos de z

Todas las anteriores son propias excepto ABC



Operaciones con Palabras (seamos formales)

Definición: Sea $a \in \Sigma$, $n \in \mathbb{N}$. Definimos la **potencia n-ésima de a**, a^n por inducción como

a) $a^0 = \lambda$

b) $a^1 = a$

c) $a^{i+1} = a^i a = a a^i \in \Sigma^*$

Definición:

Sea $x \in \Sigma^*$, $x = x[0]x[1] \dots x[n-1]$ Definimos la **palabra inversa** de x , y la notamos por x^{-1} , a la palabra

$$x^{-1} = x[n-1] \dots x[1]x[0]$$

Observaciones:

a) $|x^{-1}| = |x| \quad \forall x \in \Sigma^*$

b) $\lambda^{-1} = \lambda$



Operaciones con Palabras (seamos formales)

- Definición: Dadas dos palabras $x, y \in \Sigma^*$, se llama **concatenación de x e y** a la palabra ω que se obtiene de escribir y a continuación de x, es decir,

$$\omega = xy \text{ (o bien } \omega = x \cdot y \text{)}.$$

- Propiedades de la concatenación de palabras:

a) La concatenación es una aplicación

$$\cdot : \Sigma^* \times \Sigma^* \rightarrow \Sigma^* \quad \textbf{(operación cerrada)}$$

$$b) \quad \forall x, y, z \in \Sigma^*; (x \cdot y) \cdot z = x \cdot (y \cdot z)$$

(asociativa)

$$c) \quad \exists u \in \Sigma^*, \forall x \in \Sigma^*: u \cdot x = x \cdot u = x$$

(elemento

neutro)

(Observación: el elemento neutro es $\lambda \forall \Sigma$)

Luego $(\Sigma^*, \cdot)$ es un **semigrupo con elemento neutro o monoide libre**.



Operaciones con Palabras (seamos formales)

- Observación: Es claro que no se cumple la propiedad conmutativa

Todo monoide libre cumple la “ley de cancelación por la izquierda”, pues $\forall a, b, c \in \Sigma^*$ es cierto que:

$$ab = ac \Rightarrow b = c$$

En nuestro caso también se cumple la “ley de cancelación derecha”:

$$ba = ca \Rightarrow b = c$$



Operaciones con Palabras (seamos formales)

- Teorema 1.9: Sea Σ alfabeto (i.e., con un número finito de símbolos), entonces Σ^* es numerable.

- Demostración:

$$\Sigma = \{a_1, \dots, a_n\} \qquad |\Sigma| = n \text{ finito}$$

Σ^* es claramente infinito. Habrá que probar que existe una aplicación $f: \Sigma^* \rightarrow \mathbb{N}$ inyectiva. Pero claramente la función definida como

a) $f(\lambda) = 0$

b) $f(a_i) = i, i \leq n$

c) $f(x) = f(ya_j) = f(y)n + j \qquad \text{con } j \leq n$

es inyectiva (demostración como ejercicio).



Operaciones con Palabras (seamos formales)

- Definición 1.30: (Orden parcial)

Definimos la relación $\sqsubseteq$ entre elementos de Σ^* como

$$x \sqsubseteq y \Leftrightarrow \exists z \in \Sigma^* / y = x \cdot z \text{ (i. e. } x \text{ es prefijo de } y)$$

$$x \subset y \Leftrightarrow x \sqsubseteq y \quad \wedge \quad x \neq y$$

Definición 1.31: (orden lexicográfico. Orden total)

(Sea $f: \Sigma^* \rightarrow \mathbb{N}$ $\omega_1 < \omega_2 \Leftrightarrow f(\omega_1) <_{\mathbb{N}} f(\omega_2)$)

Sean $x, y \in \Sigma^*$, $|\Sigma| = n$ $\Sigma = \{a_1, a_2, \dots, a_n\}$

$$x \leq y :\Leftrightarrow x \sqsubseteq y \vee (\exists u, v, w \in \Sigma^* \wedge \exists i, j \in \{1, 2, \dots, n\} \text{ ,, } \\ x = ua_i v \wedge y = ua_j w \wedge i < j)$$



Lenguajes

- Definición (recordatorio): Llamaremos **lenguaje sobre un alfabeto** Σ a cualquier subconjunto del discurso universal
 $L \subseteq \Sigma^*$
- El conjunto vacío $\emptyset$ es un lenguaje sobre Σ . A este lenguaje lo llamaremos **lenguaje vacío**.
- Este lenguaje vacío **no hay que confundirlo** con el lenguaje formado por la palabra vacía, $\{\lambda\}$
- $\emptyset$ y $\{\lambda\}$ son lenguajes sobre cualquier alfabeto Σ
- El alfabeto es un lenguaje $\Sigma \subseteq \Sigma^*$



Operaciones con Lenguajes

- Las operaciones que más nos interesan sobre un lenguaje son: **Unión, Concatenación, Clausura Positiva y la Clausura o (Cierre de Kleene)**. podemos definir otras operaciones como la Potencia, Reflexión, Resta o Complemento.
- **Unión:** Sean L_1 y L_2 dos lenguajes sobre un alfabeto Σ , $L_1 \cup L_2$ es el lenguaje formado por las palabras de L_1 y por las palabras de L_2 .
 - Ejemplo: Supongamos el alfabeto $\Sigma_1 = (a, b, c, d, e, f, g, h, i)$ y dos lenguajes de ese alfabeto podrían ser: $L_1 = \{\lambda, aa, bb, cc, dd\}$ y $L_2 = \{ba, ca, da, fa, ga, ha\}$. La unión de L_1 con L_2 sería:
 - $L_1 \cup L_2 = \{\lambda, aa, bb, cc, dd, ba, ca, da, fa, ga, ha\}$, que cumple con la propiedad conmutativa, es decir que $L_2 \cup L_1$ daría el mismo resultado.



Operaciones con Lenguajes

- **Concatenación:** Sean L_1 y L_2 dos lenguajes sobre un alfabeto Σ , $L_1 \cdot L_2$ es el lenguaje formado por las palabras de L_1 concatenado con las palabras de L_2 .
 - Ejemplo: A partir del alfabeto y los lenguajes descritos en el párrafo anterior, $L_1 \cdot L_2 = \{ba, ca, da, fa, ga, ha, aaba, aaca, aada, \dots\}$.
- **Cierre Positivo (Σ^+):** Es el lenguaje formado por todas las palabras que pueden formarse con símbolos del alfabeto Σ , **excluyendo la palabra vacía**.
 - Si $L_2(\in \Sigma_1) = \{ba, ca, da\}$, el cierre positivo de ese lenguaje será $L_2^+ = \{ba, baca, bada, ca, caba, cada, \dots\}$
- **Cierre o cerradura de Kleene (Σ^*):** Es el lenguaje formado por todas las palabras que pueden formarse con símbolos del alfabeto Σ , **incluyendo la palabra vacía**, que siempre será parte del cierre.
 - Si $L_2 \in \Sigma_1 = \{ba, ca, da\}$, el cierre de ese lenguaje será $L_2^* = \{\lambda, ba, baca, bada, ca, caba, cada, \dots\}$.



Operaciones con Lenguajes (formal)

a) **Unión de lenguajes (recodatorio).** Sean L_1 , L_2 y L_3 lenguajes sobre un mismo alfabeto Σ .

$$L_1 \cup L_2 = \{x \in \Sigma^* / x \in L_1 \vee x \in L_2\}$$

Propiedades

i) La unión de lenguajes es una **Operación Cerrada**,

$$L_1 \cup L_2 \subseteq \Sigma^* \Rightarrow L_1 \cup L_2 \text{ es un lenguaje}$$

ii) **Propiedad Asociativa**

$$(L_1 \cup L_2) \cup L_3 = L_1 \cup (L_2 \cup L_3)$$

iii) **Elemento Neutro**

$$\forall L \text{ lenguaje } L \cup \emptyset = \emptyset \cup L = L$$

iv) **Conmutativa**

$$L_1 \cup L_2 = L_2 \cup L_1$$

v) **Propiedad idempotente**

$$L \cup L = L$$



Operaciones con Lenguajes (formal)

b) Concatenación de lenguajes

Definición

Sean L_1, L_2 definidos sobre el mismo alfabeto.

Llamaremos **concatenación o producto de los lenguaje**, al conjunto

$$L = L_1 L_2 = \{x \cdot y / x \in L_1 \wedge y \in L_2 \}$$

Extenderemos la definición al lenguaje vacío añadiendo que

$$\forall L \text{ lenguaje, } \emptyset L = L \emptyset = \emptyset$$



Operaciones con Lenguajes (formal)

Propiedades

i) La concatenación de lenguajes es una **Operación Cerrada**

$$L = L_1 L_2 \text{ es un lenguaje}$$

ii) **Propiedad Asociativa**

$$(L_1 L_2) L_3 = L_1 (L_2 L_3)$$

iii) **Elemento neutro**

$$\{\lambda\} L = L \{\lambda\} = L \quad \forall L$$

Por cumplir i), ii) y iii) la concatenación de lenguajes sobre un mismo alfabeto es un monoide.



Operaciones con Lenguajes (formal)

- Teorema: Dados los lenguajes A, B y C sobre el alfabeto Σ , se cumple que:

i) $A \bullet (B \cup C) = A \bullet B \cup A \bullet C$

ii) $(B \cup C) \bullet A = B \bullet A \cup C \bullet A$

Demostración: Ejercicio.

Potencia de un lenguaje

Sea L un lenguaje. Definimos **potencia i-ésima de L** al resultado de concatenarlo consigo mismo i veces:

- $L^i = LL\dots L$ (i veces) para $i > 1$
- Si $i=1$

$$L^1 = L$$

- $\forall i, j > 0$ se verifica que

$$L^{i+1} = L^i L = L L^i$$

$$L^i L^j = L^{i+j}$$

$$L^0 = \{\lambda\} \forall L \subseteq \Sigma^*$$



Operaciones con Lenguajes (formal)

d) Clausuras de un lenguaje.

Definiremos la **clausura de un lenguaje L**, y lo denotaremos por L^* , como

$$L^* = \bigcup_{i=0}^{\infty} L^i \text{ (cierre de Kleene)}$$

- Definiremos la **clausura positiva de un lenguaje L**, y lo denotaremos por L^+ , como

$$L^+ = \bigcup_{i=1}^{\infty} L^i \text{ (cierre positivo de Kleene)}$$

Teorema: Sea $L \subseteq \Sigma^*$, entonces

i) $L^* = L^+ \cup \{\lambda\}$

ii) $L^+ = LL^* = L^*L$

PREGUNTA DE EXAMEN: ¿ $L^+ = L^* \setminus \{\lambda\}$ siempre?



Operaciones con Lenguajes (formal)

e) Reflexión de lenguajes.

Sea L un lenguaje cualquiera. Se llama **lenguaje reflejo** o **inverso** de L , y se representa por L^{-1} a

$$L^{-1} = \{x^{-1} / x \in L\}$$

Observaciones:

– L^{-1} es el lenguaje formado por las palabras inversas a las de L .

– $L^{-1-1} = L \quad \forall L \subseteq \Sigma^*$

- Teorema: Sean A, B lenguajes sobre un mismo alfabeto. Entonces

$$(A \bullet B)^{-1} = B^{-1} \bullet A^{-1}$$



Operaciones con Lenguajes (formal)

f) Otras operaciones

Pueden definirse con los lenguajes todas las operaciones propias de los conjuntos.

Ejemplo:

- Intersección de lenguajes
- Complementación (con respecto al lenguaje universal).

También pueden definirse operaciones entre lenguajes de distintos alfabetos.

Ejemplo: Traducciones



Lenguajes sobre distintos alfabetos.

Homomorfismos

Definición:

Diremos que una aplicación $f: \mathbb{A}^* \rightarrow \mathbb{B}^*$ es un **homomorfismo** si:

a) $f(\lambda) = \lambda$

b) $f(xy) = f(x)f(y)$



Lenguajes sobre distintos alfabetos.

Homomorfismos

Sea $f: \mathbb{A}^* \rightarrow \mathbb{B}^*$ un homomorfismo y sea $L \subseteq \mathbb{A}^*$ lenguaje.

- Se define el lenguaje inducido por L vía f en $\mathbb{B}^*$ y se denota por $f(L)$ por

$$f(L) = \{w \in \mathbb{B}^* / \exists z \in L \text{ (subconjunto de } \mathbb{A}^* \text{)}, f(z) = w\} \subseteq \mathbb{B}^*$$

- Sea $L \subseteq \mathbb{B}^*$, podemos definir el lenguaje inverso de L (lenguaje de $\mathbb{B}^*$) respecto de f y se denota por $f^{-1}(L)$ como

$$f^{-1}(L) = \{z \in \mathbb{A}^* / \exists w \in L, f(z) = w\}$$



Operaciones entre lenguajes

Definición 1.38: Sean Σ y Ψ alfabetos y sea

$$f: \Sigma \rightarrow \wp(\Psi^*)$$

$$a \rightarrow f(a) = L_a \subseteq \Psi^*$$

se dice entonces que f es una **sustitución**.

- Podemos extender la función f para que su dominio sea:

- $\text{Dom}(f) = \Sigma^*$

$$f: \Sigma^* \rightarrow \wp(\Psi^*)$$

- $\text{Dom}(f) = \wp(\Sigma^*)$

$$f: \wp(\Sigma^*) \rightarrow \wp(\Psi^*)$$



Operaciones entre lenguajes

- Definición 1.39: Sean Σ y Ψ dos alfabetos y sea $w = w[0]w[1] \dots w[n-1]$ una palabra sobre Σ^* .

Entonces, podemos definir:

$$a) \bar{f}: \Sigma^* \rightarrow \wp(\Psi^*)$$

$$w \rightarrow \bar{f}(w) = L_w = L_{w[0]}L_{w[1]} \dots L_{w[n-1]}$$

$$b) \bar{f}: \wp(\Sigma^*) \rightarrow \wp(\Psi^*)$$

$$L \rightarrow \bar{f}(L) = \bigcup_{w \in L} L_w$$



Operaciones entre lenguajes

- Ejemplo:

Sean $\Sigma = \{0,1\}$ $\Psi = \{a,b\}$

Definimos la sustitución:

$f: \Sigma \rightarrow \wp(\Psi^*)$ tal que:

$$f(0) = \{a\}$$

$$f(1) = \{\lambda, b, bb, \dots\}$$

a) $\overline{f}: \Sigma^* \rightarrow \wp(\Psi^*)$

$$\overline{f}(010) = f(0)f(1)f(0) = \{ab^ja, j \in \mathbb{N}\}$$

b) $\overline{\overline{f}}: \wp(\Sigma^*) \rightarrow \wp(\Psi^*)$

$$\overline{\overline{f}}(\{0^i1 : i \in \mathbb{N}\}) = \{a^ib^j : i,j \in \mathbb{N}\}$$



Repaso y preguntas con intención...

1. ¿Cuántas letras hay en la palabra FABADA?

1. Demuestre el siguiente teorema:

Teorema: Sean A,B lenguajes sobre un mismo alfabeto. Entonces

$$(A \bullet B)^{-1} = B^{-1} \bullet A^{-1}$$

3. ¿qué tiene que ocurrir para que se cumpla la siguiente igualdad?

$$L^* = L^+$$

